Arquitectura del sistema — Tópicos, GNFC y WHCA*

Mapa de tópicos

Infraestructura compartida (una instancia para todos los robots)

Tópico	Tipo	Quién publica	Quién suscribe	Descripción
`/map`	`OccupancyGrid`	`map_server`	`cost_map`, RViz	Mapa estático del escenario
`/inflated_map`	`OccupancyGrid`	`cost_map`	`a_star`, `whca_star`, RViz	Mapa con obstáculos inflados por radio del robot (0.5 m)
`/cost_map`	`OccupancyGrid`	`cost_map`	`a_star`, `whca_star`, RViz	Mapa con gradiente de costo alrededor de obstáculos
`/replan_needed`	`Bool`	`cost_map`	`pure_pursuit` (todos)	Se activa cuando un robot muere y el mapa se reconstruye
`/space_time_reservations`	`SpaceTimeReservation`	`whca_star` (c/robot)	`whca_star` (c/robot)	Reservaciones espacio-temporales de cada robot
`/robot_priorities`	`String`	`priority_manager`	`whca_star` (c/robot)	Orden de prioridad: `"alvin,simon,teodoro"`
`/swarm/goal_areas`	`MarkerArray`	`swarm_viz_bridge`	RViz	Zonas de meta visualizadas como cajas verdes en RViz
`/goal_zone_marker`	`Marker`	`goal_zone_visualizer`	RViz	Rectángulo verde de zona de meta (lanzado aparte)
`/clock`	`Clock`	Gazebo	Todos los nodos con `use_sim_time`	Tiempo de simulación
`/tf`, `/tf_static`	`TFMessage`	Gazebo, `robot_state_publisher`	`pure_pursuit`, RViz	Árbol de transformadas (pose de cada robot)

Por robot — ejemplo con `/alvin/` (igual para `/teodoro/` y `/simon/`)

Tópico	Tipo	Quién publica	Quién suscribe	Descripción
`/alvin/goal_pose`	`PoseStamped`	GNFC allocator / usuario	`pure_pursuit`, `whca_coordinator`, `priority_manager`	Meta de navegación del robot

Tópico	Tipo	Quién publica	Quién suscribe	Descripción
`/alvin/cmd_vel`	`Twist`	`pure_pursuit`	Gazebo (puente ROS-GZ)	Comandos de velocidad (v, ω)
`/alvin/odom`	`Odometry`	Gazebo	`swarm_viz_bridge`, GNFC	Odometría local (origen = spawn del robot)
`/alvin/odom_map`	`Odometry`	`swarm_viz_bridge`	RViz, `swarm_health_monitor`	Odometría corregida al frame `map`
`/alvin/path_planning/path`	`Path`	`whca_star` / `a_star`	RViz	Trayectoria planificada (para visualización)
`/alvin/path_planning/smoothed_path`	`Path`	`path_smoothing`	RViz	Trayectoria suavizada
`/alvin/health`	`RobotHealth`	`swarm_health_monitor`	`cost_map`, `priority_manager`	Estado del robot: posición, velocidad, alive/dead
`/alvin/kill`	`Bool`	GUI (teleop)	`pure_pursuit`, `swarm_health_monitor`	Simula muerte del robot
`/alvin/navigation/goal_reached`	`Bool`	`pure_pursuit`	`gnfc_goal_allocator`, `whca_coordinator`	Señal de meta alcanzada
`/alvin/whca_replan`	`Bool`	`whca_coordinator`	`pure_pursuit`	Solicitud de replanning desde el coordinador WHCA*
`/alvin/robot_description`	`String`	`robot_state_publisher`	RViz	URDF del robot para visualización
`/alvin/joint_states`	`JointState`	Gazebo	`robot_state_publisher`	Estado de las ruedas

Cómo funciona GNFC (Greedy Nearest-Free-Cell)

Problema que resuelve: asignar una meta única dentro de una zona rectangular a cada robot, sin que dos robots vayan al mismo punto.

Algoritmo

```
1. Generar grilla de candidatos dentro de [x_min, x_max] × [y_min, y_max] con spacing dado
2. Para cada robot que aún no tiene meta:
   a. Calcular distancia de ese robot a todos los candidatos disponibles
   b. Asignar el candidato más cercano
   c. Remover ese candidato del pool (ya no disponible para otros robots)
3. Publicar PoseStamped a /{robot}/goal_pose para cada asignación
4. No volver a asignar hasta que el robot publique goal_reached
```

Propiedades

- **Complejidad:** O(n·m) — n robots, m candidatos en la grilla
- **Greedy:** no garantiza el óptimo global, pero es rápido y suficiente en práctica
- **Asignación estable:** una vez asignado, el goal no cambia hasta que el robot llega

Flujo de datos

```
GNFC lee /{robot}/odom
  → calcula distancias a candidatos libres
  → asigna el más cercano a cada robot que lo necesite
  → publica /{robot}/goal_pose
        ↓
  pure_pursuit recibe goal → planea → sigue ruta
        ↓
  pure_pursuit publica /{robot}/navigation/goal_reached
        ↓
  GNFC reasigna solo ese robot (los demás conservan su meta)
```

Cómo funciona WHCA *(Windowed Hierarchical Cooperative A)*

Problema que resuelve: planificar rutas para múltiples robots que no se colisionen entre sí, incluso cuando sus trayectorias convergen o se cruzan.

Componentes

Nodo	Función
`priority_manager`	Ordena robots por distancia a su meta (mayor distancia = mayor prioridad)
`whca_star` (×3)	Planificador espacio-temporal por robot, sirve el servicio `plan_path`
`whca_coordinator`	Dispara replanning periódico, escalonado por prioridad

Espacio de búsqueda: 2D → 3D

En A* estándar el estado es `(fila, columna)`.

En WHCA *el estado es `(fila, columna, paso_de_tiempo)`. Una celda está "bloqueada" no solo si hay un obstáculo estático, sino también si un robot de mayor prioridad planea ocupar esa celda en ese paso de tiempo*.

```
Tiempo →   t=0  t=1  t=2  t=3  t=4
Alvin       ●    ●    ●    ●    ●    ← reservaciones publicadas
Simon            ✗    ✗    ✗         ← Simon no puede ir ahí en esos pasos
```

Ventana deslizante (Windowed)

El planificador calcula la ruta completa hasta la meta, pero solo reserva los primeros **W pasos** (`window_size = 40 celdas ≈ 2 m`). Esto:

- Mantiene el espacio de búsqueda manejable
- Permite actualizar reservaciones con información reciente
- El coordinador dispara `/{robot}/whca_replan` cada **5 segundos** para deslizar la ventana

---

Flujo de planificación — una ronda de replan

```
whca_coordinator dispara replan (escalonado por prioridad)
  │
  ├─→ /alvin/whca_replan    (prioridad 1, delay = 0.0 s)
  ├─→ /simon/whca_replan    (prioridad 2, delay = 0.8 s)
  └─→ /teodoro/whca_replan  (prioridad 3, delay = 1.6 s)

── Alvin (prioridad más alta) ──
  pure_pursuit recibe whca_replan → pide plan a whca_star
  whca_star: blocked = {} (nadie tiene mayor prioridad)
  → planifica ruta completa a meta con A* espacio-temporal
  → publica reservaciones en /space_time_reservations (primeros 40 pasos)
  → pure_pursuit sigue la ruta

── Simon (0.8 s después) ──
  whca_star: blocked = reservaciones de Alvin (infladas ±5 celdas)
  → planifica ruta evitando celdas reservadas por Alvin
  → publica sus propias reservaciones

── Teodoro (1.6 s después) ──
  whca_star: blocked = reservaciones de Alvin + Simon
  → planifica evitando a ambos
```

Margen de seguridad — inflación de reservaciones

Cada celda reservada `(r, c, t)` se infla a todos sus vecinos dentro de ±5 celdas antes de agregarse al blocked set del robot de menor prioridad:

```
Radio físico del robot  ≈ 0.25 m
Resolución del mapa     = 0.05 m/celda
SAFETY_CELLS            = ⌈0.25 / 0.05⌉ = 5 celdas

Clearance total entre centros = 2 × 5 × 0.05 = 0.50 m
```

Esto es consistente con el `inflation_radius = 0.5 m` usado en el costmap estático y satisface el criterio cinemático: `d_safe ≥ 2 × r_robot` [Silver 2005].

Manejo de robots muertos

Cuando un robot es marcado como muerto por el health monitor:

```
swarm_health_monitor detecta inmovilidad > 3 s
  → publica /{robot}/health (alive = false)
        ↓
  cost_map marca última posición como obstáculo permanente
  → reconstruye inflated_map y cost_map
  → publica /replan_needed (todos los pure_pursuit replanean)
        ↓
  El robot muerto deja de publicar /space_time_reservations
        ↓
  En whca_star: reservaciones tienen TTL = 12 s
  → después de 12 s, sus entradas expiran automáticamente
  → supervivientes pueden planear libremente por esas celdas
  (el obstáculo estático en el mapa ya garantiza evasión física)
```

Diagrama completo de tópicos WHCA*

```
                    /robot_priorities
priority_manager ──────────────────────────────────→ whca_star[*]
                                                        ↑↓
gnfc_allocator ──→ /{robot}/goal_pose          /space_time_reservations
                          ↓
whca_coordinator ──→ /{robot}/whca_replan
                          ↓
                   pure_pursuit[robot]
                    │
                    ├─ SM_WAIT_FOR_NEW_GOAL  ←── /{robot}/goal_pose
                    ├─ SM_PLAN_PATH          ──→ plan_path service ──→ whca_star
                    ├─ SM_SMOOTH_PATH        ──→ smooth_path service
                    ├─ SM_FOLLOWING_PATH     ──→ /{robot}/cmd_vel
                    └─ SM_SAVE_DATA          ──→ /{robot}/navigation/goal_reached
                                                        ↓
                                              gnfc_allocator reasigna
                                              whca_coordinator deja de replanear
```